Numerical astrophysics WS 2025

Mini-projects

The mini-projects are worth 25% of the total grade of the course (25 points maximum). The deadline for submission of mini-projects is February, 15th. Mini-projects should be uploaded to Github. The teaching team should then be added to the repositories as collaborators. The students should select **one mini-project out of the three** presented below. The mini-projects are individual projects (no group work).

1 Project 1. Classes & dictionaries

Write a code which calculates the optical depth and the transmissivity at a given distance R from a planet's radius and for a given wavelength λ given in μm (see Fig. 1 for the geometry). Cross section data $\sigma(\nu)$ can be found in the data file CO2.dat. Pay attention to the units used in the file (the difference of ν and λ and the fact that the unit of σ is cm^2/molecule). Use at least one class and a parameter file with parameters read and stored as a dictionary (parameters such as the wavelength λ and the radius R of interest, the name of the cross section file, the atmospheric density at the surface ρ_{surf}, and the temperature T_{atm}, the planet's mass and radius M_{pl} and R_{pl}). Write this code as a module which can be loaded using "import module_name" and then used like module_name.opticaldepth(λ, R) (you can give the functions and the module the names

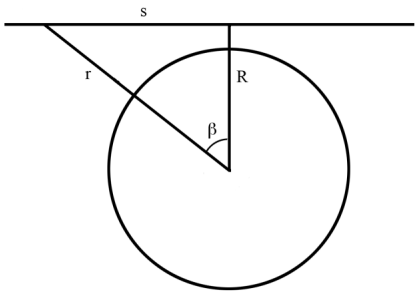

Figure 1: *The geometry of the problem for mini-project 1.*

you like).

To calculate the optical depth for a given frequency at a given distance from the planet, use the formulas from the radiative transfer equations. The solution for the integrated column density of an atmospheric gas vs R is given by $N_t(R) = 2N(R)Re^{R/H}K_1(R/H)$, where K_1 is the modified Bessel function of the third kind. You can also use a simpler approximation

$$N_t(R) \approx N(R)(2\pi RH)^{1/2}. \tag{1}$$

You can use the barometric formula for $N(R) = N_{surf}\exp(-h/H)$, where $H = R_g T_{atm}/m_{gas}g$ is the scale height and h is the altitude, $h = R - R_{pl}$. Here, R_g is the gas constant, T_{atm} is the atmosphere's temperature, m_{gas} is the mass of a molecule (assume CO_2 for your assignment), and g is the gravitational acceleration. Knowing the column density, you can use the Beer-Lambert law to calculate the optical depth $\tau(\lambda) = N_t\sigma(\lambda)$ and the transmissivity $T(\lambda) = e^{-\tau(\lambda)}$.

For more details, see the file AT1.pdf, and the radiative transfer equations from the theoretical courses and/or text books.

2 Project 2. Visualisation

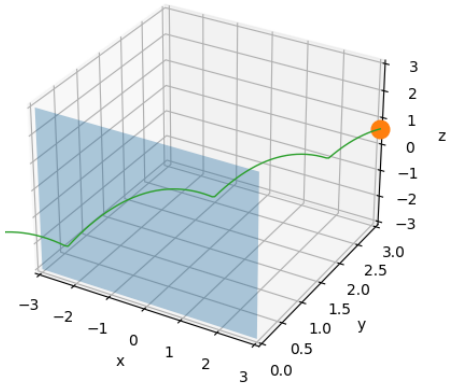

Figure 2: *An example visualisation not including the proper GUI.*

Write a code which calculates how a ball falls, if dropped at a height H with initial

velocity components v_x, v_y, v_z. Calculate where it lands. Make a GUI where one can put in H and velocity components, which then executes the code and animates the ball's trajectory and shows the landing spot. For simplicity, consider that the ball falls on a flat surface, and the gravitational acceleration equals that of the Earth. The equations describing the motion of the ball are those of the accelerated motion with an initial velocity in 3D.

3 Project 3. Fitting & testing

Write a code which fits the depth and the length of an exoplanetary transit. A transit is an event when an exoplanet passes in front of its host star as seen from the Earth, which causes the apparent dimming in the stellar light. Usually, multiple transits are observed until the presence of an exoplanet is confirmed, but in this simple example we consider a single transit (Fig. 3).

- Load the transit data from the file ArtificialTransitData.txt into your code.
- Generate your own transit, and move it along the time line as shown in Fig 3 with the red line.
- Calculate the χ^2. Vary the transit depth and the transit length and fit the best pair which produces the smallest χ^2.
- Benchmark the code.
- Use pytest for testing your code. Assert that *i)* the fitted depth is close to the true injected depth (within 15%); *ii)* test the behaviour when pure noise (no transit) into the fitter and check that your programme doesn't "imagine" a deep signal (check for false positives - for this text, you will need to add a small random noise to your generated transit); *iii)* check that the code does not generate negative transit duration and depth (physics sanity check).

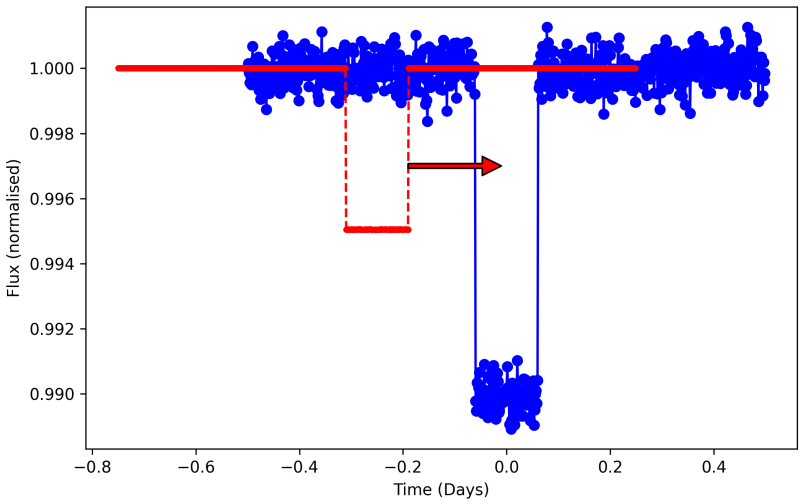

Figure 3: *A simplified representation of a single transit of an exoplanet. The data shown with blue dots can be found in the file ArtificialTransitData.txt. The data shown in red represent the transit generated by your programme.*